

Savitribai Phule Pune University

Fourth Year of Computer Engineering (2019 Course)

410256: Laboratory Practice VI

**Elective III: 410252(A) Natural Language
Processing**

Mini-Project Report

Part-of-Speech Tagging for Indian Languages
(Marathi)

Using Conditional Random Fields

Submitted By:

Name	Roll number
Aharnish Parekar	3446
Sudesh Nawale	3442
Piyush Ovhal	3444
Himanshu Dhayarkar	3415
Chirag Attar	3402

Department of Computer Engineering

Contents

1	Objectives	2
2	Problem Statement	2
3	Requirements	2
	Requirements	2
	Software Requirements	2
4	Theory and Concept in Brief	3
	4.1 Introduction to POS Tagging	3
	4.2 Challenges in Indian Languages	3
	4.3 Conditional Random Fields (CRF)	3
5	System Architecture and Feature Engineering	
	5.1 Dataset Preparation	
	5.2 Feature Extraction Strategy	
		4
		4
		4
6	Algorithm and Implementation	4
7	Test Cases and Analysis	
	7.1 Quantitative Analysis	
	7.2 Qualitative Analysis (Sample Output)	
		5
		5
		5
8	Conclusion	6

1 Objectives

- To understand the fundamental concepts of Natural Language Processing (NLP) specifically tailored for morphologically rich Indian languages.
- To explore and implement state-of-the-art sequence labeling algorithms.
- To design and develop a Part-of-Speech (POS) Tagger for the Marathi language using Conditional Random Fields (CRFs).
- To evaluate the performance of the statistical model using standard classification metrics.

2 Problem Statement

Design and implement a machine learning model to accurately predict the Part-of-Speech (POS) tags for sentences in the Marathi language. The system should utilize the NLTK Indian Corpus for training and employ a Conditional Random Field (CRF) algorithm to handle the sequential and morphologically complex nature of Marathi text, ultimately demonstrating feature extraction techniques and model evaluation.

3 Requirements

3.1 Hardware Requirements

- **Processor:** 64-bit multi-core processor (Intel i5/AMD Ryzen 5 or higher recommended).
- **Memory:** 8 GB RAM minimum.
- **Storage:** 10 GB available disk space.
- **Accelerator (Optional):** Cloud-based GPU via Google Colab (NVIDIA T4).

3.2 Software Requirements

- **Operating System:** 64-bit Open Source Linux or its derivative / Windows / macOS.
- **Programming Language:** Python 3.8+
- **Libraries:** NLTK, scikit-learn, sklearn-crfsuite.
- **Environment:** Google Colaboratory / Jupyter Notebook.

4 Theory and Concept in Brief

4.1 Introduction to POS Tagging

Part-of-Speech (POS) tagging is the process of marking up a word in a text as corresponding to a particular part of speech, based on both its definition and its context. In languages like English, structural word order heavily dictates tagging. However, Indian languages like Marathi are highly inflectional and have a relatively free word order. This makes rule-based tagging exceptionally difficult and necessitates the use of statistical and machine learning approaches.

4.2 Challenges in Indian Languages

Marathi is an Indo-Aryan language that is morphologically rich. A single root word can take dozens of forms depending on gender, number, person, and tense. Suffixes are frequently attached to root words to denote prepositions or conjunctions. Therefore, a model must look at sub-word features (like prefixes and suffixes) rather than just the whole word.

4.3 Conditional Random Fields (CRF)

A Conditional Random Field (CRF) is a standard model for predicting the most likely sequence of labels that correspond to a sequence of inputs. Unlike Hidden Markov Models (HMMs), which are generative and assume that features are conditionally independent, CRFs are discriminative models. They model the conditional probability $P(Y|X)$ directly. The probability of a label sequence $Y = (y_1, y_2, \dots, y_N)$ given an input sequence $X = (x_1, x_2, \dots, x_N)$ is defined as:

$$P(Y|X) = \frac{1}{Z(X)} \exp \left(\sum_{i=1}^N \sum_{j=1}^K \lambda_j f_j(y_{i-1}, y_i, X, i) \right) \quad (1)$$

Where:

- $Z(X)$ is the normalization factor.
- $f_j(y_{i-1}, y_i, X, i)$ is a feature function that looks at the current label, the previous label, the entire input sequence, and the current position.
- λ_j is the learned weight for the feature function f_j .

CRFs are considered state-of-the-art for traditional POS tagging because they avoid the label bias problem found in Maximum Entropy Markov Models (MEMMs) and can seamlessly integrate a vast array of overlapping features (e.g., capitalization, word length, suffixes).

5 System Architecture and Feature Engineering

5.1 Dataset Preparation

The model utilizes the ‘indian’ corpus provided by the Natural Language Toolkit (NLTK). This corpus contains pre-tagged sentences in several Indian languages. The Marathi subset is extracted, providing a ground-truth dataset of token-tag pairs.

5.2 Feature Extraction Strategy

To leverage the CRF architecture, raw text must be converted into a rich feature set. For each token in a sentence, the following features are extracted:

1. **Lexical Features:** The exact string of the word (lowercased).
2. **Morphological Features:** The last 2 and last 3 characters of the word. This is highly effective for Marathi as it captures grammatical suffixes indicating tense or plurality.
3. **Contextual Features:** The lexical and morphological features of the previous word ($i - 1$) and the next word ($i + 1$).
4. **Positional Features:** Boolean flags indicating if the word is at the Beginning of the Sentence (BOS) or the End of the Sentence (EOS).

6 Algorithm and Implementation

The system is implemented in Python. Below is the core logic for the feature extraction algorithm which serves as the foundation for the CRF model.

```
1 def word2features(sent, i): word =
2     sent[i][0] features = {
3         'bias': 1.0 ,
4         'word.lower()': word.lower(),
5         'word[-3:]': word[-3:],
6         'word[-2:]': word[-2:],
7         'word.isdigit()': word.isdigit(),
8     }
9     if i > 0:
10         word1 = sent[i-1][0]
11         features.update({
12             '-1: word.lower()': word1.lower(),
13             '-1: word.isdigit()': word1.isdigit(),
14         })
15     else:
16         features['BOS'] = True
```



```

18
19     if i < len(sent)-1: word1 =
20         sent[i+1][0]
21         features.update({
22             '+1: word.lower()': word1.lower(),
23             '+1: word.isdigit()': word1.isdigit(),
24         })
25     else:
26         features['EOS'] = True
27
28     return features

```

Listing 1: Feature Extraction Logic

The L-BFGS (Limited-memory Broyden-Fletcher-Goldfarb-Shanno) algorithm is used to optimize the weights (λ_j) during the training phase, with L1 and L2 regularization applied to prevent overfitting.

7 Test Cases and Analysis

The dataset was split into an 80% training set and a 20% testing set. The model was trained and subsequently evaluated against unseen data.

7.1 Quantitative Analysis

The CRF model achieved an overall accuracy of **81.95%** on the testing dataset.

7.2 Qualitative Analysis (Sample Output)

A custom sentence was passed into the trained model to verify real-world inference capabilities.

Input Sentence: तो एक चांगला मुलगा आहे (He is a good boy)

Predicted Output:

- तो → PRP (Pronoun)
- एक → QC (Quantifier)
- चांगला → JJ (Adjective)
- मुलगा → NN (Noun)
- आहे → VM (Verb)

8 Conclusion

This mini-project successfully demonstrates the application of Machine Learning in Natural Language Processing for Indian languages. By implementing a Conditional Random Field (CRF), we bypassed the limitations of traditional rule-based taggers and generative models like HMMs. The strategic extraction of morphological suffixes proved to be highly effective for Marathi's complex linguistic structure. Future scope includes expanding the dataset, integrating deep learning models like BiLSTMs, and building a web-based UI for real-time tagging.

References

- [1] Bird, Steven, Edward Loper and Ewan Klein (2009). *Natural Language Processing with Python*. O'Reilly Media Inc.
- [2] Lafferty, J., McCallum, A., & Pereira, F. C. (2001). *Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data*.


```
# Cell 1: Install dependencies and download NLTK data
```

```
!pip install sklearn-crfsuite -q
```

```
import nltk
```

```
nltk.download('indian', quiet=True)
```

```
nltk.download('punkt', quiet=True)
```

```
print("Libraries installed and NLTK corpora downloaded successfully!")
```

```
0.0/1.2 MB ? eta -:--:--
```

```
—— 1.2/1.2 MB 40.5 MB/s eta 0:00:00
```

```
# Cell 2: Load the dataset
```

```
from nltk.corpus import indian
```

```
# Load tagged sentences for Marathi
```

```
marathi_sents = indian.tagged_sents('marathi.pos')
```

```
print(f"Total sentences in the Marathi dataset: {len(marathi_sents)}") print("\nSample tagged sentence (Word, Tag):")
```

```
print(marathi_sents[0])
```

```
Total sentences in the Marathi dataset: 1197
```

```
Sample tagged sentence (Word, Tag):
```

```
[('','SYM'), ('सनाTनवाTBTनी', 'NN'), ('वा', 'CC'), ('प्रतितगTम्यांTनी', 'NN'), ('समाT', 'NN'), ('रसTतळाTलाT', 'NN'), ('नाेलाT', 'VM'), ('असतTनाT', 'VAUX'), ('यांT', 'DEM'), ('अॅरTरTत', 'NN'), ('बाTळाशाTस्त्री', 'NNPC'), ('NTभेेकर', 'NNP'), ('यांTनी', 'PRP'), ('"5पॅण"sयांT', 'NNP'), ('माTयांमाTतना', 'NN'), ('पॅतिहली', 'QO'), ('KTनाज्यांTत', 'NN'), ('तेवावाली', 'VM'), (',', 'SYM'), ('','SYM'), ('असे', 'DEM'), ('प्रतितपॅTना', 'NN'), ('नाटसेTट', 'NNPC'), ('प्रभेTकर', 'NNPC'), ('पॅणशीकर', 'NNP'), ('यांTनी', 'PRP'), ('केेलाे', 'VM'), (',', 'SYM')]
```

```
# Cell 3: Define feature extraction functions
```

```
def word2features(sent, i): word
```

```
    = sent[i][0] features = {
```

```
        'bias': 1.0,
```

```
        'word.lower()': word.lower(),
```

```
        'word[-3:]': word[-3:], # Last 3 characters (useful for Marathi suffixes)
```

```
        'word[-2:]': word[-2:], # Last 2 characters
```

```
        'word.isdigit()': word.isdigit(),
```

```
    }
```

```
    # Features for the previous word
```

```
    if i > 0:
```

```
        word1 = sent[i-1][0]
```

```
        features.update({
```

```

        '-1:word.lower()': word1.lower(),
        '-1:word.isdigit()': word1.isdigit(),
    })
else:

    features['BOS'] = True # Beginning of Sentence

    # Features for the next word
    if i < len(sent)-1: word1 =
        sent[i+1][0]
        features.update({
            '+1:word.lower()': word1.lower(),
            '+1:word.isdigit()': word1.isdigit(),
        })
    else:
        features['EOS'] = True # End of Sentence

    return features

def sent2features(sent):
    return [word2features(sent, i) for i in range(len(sent))]

def sent2labels(sent):
    return [label for token, label in sent]

# Cell 4: Train-test split
from sklearn.model_selection import train_test_split

# Prepare features (X) and labels (y)
X = [sent2features(s) for s in marathi_sents] y =
[sent2labels(s) for s in marathi_sents]

# Split into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

print(f"Training sentences: {len(X_train)}")
print(f"Testing sentences: {len(X_test)}")

Training sentences: 957
Testing sentences: 240

# Cell 5: Initialize and train the model
import sklearn_crfsuite

# Initialize the Conditional Random Field (CRF) model
crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    c1=0.1, # L1 regularization
    c2=0.1, # L2 regularization
    max_iterations=100,
    all_possible_transitions=True

```

```

)

# Train the model
print("Training the CRF model...")
crf.fit(X_train, y_train) print("Training
complete!")

Training the CRF model...
Training complete!

# Cell 6: Calculate accuracy and
metrics from sklearn.metrics import
accuracy_score from sklearn_crfsuite import
metrics import warnings
warnings.filterwarnings('ignore') # Ignore zero-division warnings for
sparse tags

# Predict on the test set
y_pred = crf.predict(X_test)

# Flatten the lists for standard accuracy calculation
flat_y_test = [label for sent in y_test for label in sent] flat_y_pred = [label
for sent in y_pred for label in sent]

accuracy = accuracy_score(flat_y_test, flat_y_pred) print(f"Model Overall
Accuracy: {accuracy * 100:.2f}%\n")

# Display detailed classification
report labels = list(crf.classes_)
print("Detailed Classification Report:")
print(metrics.flat_classification_report(y_test, y_pred, labels=labels))

```

Model Overall Accuracy: 81.95%

Detailed Classification Report:

	precision	recall	f1-score	support
NNP	0.74	0.58	0.65	339
VM	0.89	0.86	0.88	588
NNPC	0.58	0.77	0.66	225
JJ	0.77	0.67	0.72	234
NN	0.75	0.89	0.81	1098
PRP	0.90	0.77	0.83	302
SYM	1.00	0.97	0.98	517
DEM	0.86	0.92	0.89	104
QO	1.00	0.75	0.86	8
WQ	1.00	0.75	0.86	12
VAUX	0.88	0.91	0.90	218
NST	1.00	0.33	0.50	24
CC	0.93	0.97	0.95	131

	0.00	0.00	0.00	0
QC	0.89	0.87	0.88	112
RB	0.87	0.42	0.57	31
PSP	0.50	0.50	0.50	2
QF	0.93	0.60	0.73	67
NNC	0.63	0.42	0.50	77
NEG	1.00	1.00	1.00	3
INTF	0.00	0.00	0.00	6
UNK	1.00	0.60	0.75	5
RP	1.00	0.30	0.46	10
UT	0.00	0.00	0.00	1
RDP	0.00	0.00	0.00	1
INJ	0.00	0.00	0.00	1
SYMC	0.00	0.00	0.00	0
"CC	0.00	0.00	0.00	0

accuracy			0.82	4116
macro avg	0.65	0.53	0.57	4116
weighted avg	0.83	0.82	0.82	4116

```
# Cell 7: Test with a custom sentence
```

```
# Custom Marathi sentence: "त॒त॒ ड॒क चां॑गला॒त॒ मा॒लाग॑त॒ न्ह॑ " (He is a good boy)
```

```
custom_sentence = "त॒त॒ ड॒क चां॑गला॒त॒ मा॒लाग॑त॒ न्ह॑ "
custom_tokens = custom_sentence.split()
```

```
# Create dummy tags to reuse our feature extraction structure
dummy_sent = [(word, "") for word in custom_tokens]
custom_features = [sent2features(dummy_sent)]
```

```
# Predict tags
custom_prediction = crf.predict(custom_features)[0]
```

```
print(f"Input Sentence: {custom_sentence}")
print("-" * 30)
print("Predicted POS Tags:")
for word, tag in zip(custom_tokens, custom_prediction): print(f"{word} -> {tag}")
```

Input Sentence: त॒त॒ ड॒क चां॑गला॒त॒ मा॒लाग॑त॒ न्ह॑

Predicted POS Tags:

त॒त॒ -> PRP

ड॒क -> QC

चां॑गला॒त॒

-> JJ

मा॒लाग॑त॒

-> NN

न्ह॑ ->

VM